

Esame OS Internals (Cabodi)

Data: 09/06/2021

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N (128*1024)
int i, j;
int A[1024], B[N];

for (j=0; j<N; j++) {
    B[j] = 0;
}

for (i=0; i<1024; i++) {
    j = A[i];
    if (j>=0 && j<N)
        B[j]++;
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con memoria virtuale gestita mediante paginazione, con pagine di 2 KByte, utilizzando come politica di sostituzione pagine la SECOND CHANCE. Si sa che un `int` ha dimensione 32 bit e che le istruzioni in codice macchina sono contenute in una sola pagina.

Si risponda alle seguenti domande:

-) Quante pagine (e frame) sono necessarie per contenere i vettori **A** e **B**?
-) Ipotizzando che le variabili **i**, **j** siano allocate in registri della CPU, quanti accessi in memoria (in lettura e scrittura) fa il programma proposto per accedere a dati (non vanno conteggiati gli accessi a istruzioni)? (Calcolare sia il caso minimo che il caso massimo).
-) Calcolare il numero di page fault generati dal programma proposto, supponendo che siano allocati per esso 10 frame, di cui uno utilizzato (già all'inizio dell'esecuzione) per le istruzioni. (Considerare caso minimo e massimo, e motivare le risposte).

Domanda 2 (SSD e Wear Leveling)

Contesto: Sia dato un disco di tipo "solid state" (SSD). Una delle tecniche usate per prolungare la vita di un dispositivo SSD è il cosiddetto "wear leveling", che consiste nel mantenere una tabella (mappa) di corrispondenza tra indirizzi logici e fisici per i blocchi su disco.

Si risponda alle seguenti domande:

-) Perché è opportuna questa operazione?
 -) Da cosa nasce il possibile vantaggio, da un numero minore complessivo di scritture su disco, oppure da qualcos'altro?
 -) E' possibile che un disco SSD continui a funzionare correttamente una volta che una o più pagine (blocchi) fisiche siano marcate come inutilizzabili (in quanto hanno raggiunto il numero massimo di riscritture)? (Se sì, spiegare come; se no, motivare).
-

Domanda 3 (I/O Sincrono, Asincrono e Bloccante)

Contesto: Si considerino i due tipi di sincronizzazione, relativi a operazioni di I/O: sincrono e asincrono.

Si risponda alle seguenti domande:

-) Si spieghino le principali differenze tra gli I/O dei due tipi. Si considerino poi I/O bloccante e non bloccante: si tratta di sinonimi di asincrono e sincrono? Ci sono differenze? (se no, perché; se sì, quali?).
-) Un I/O sincrono può essere effettuato in polling oppure è necessario effettuarlo in interrupt? (Si risponda alla stessa domanda per il caso dell'I/O asincrono).
-) In quale modo può un processo beneficiare di un I/O asincrono? E' possibile, nel caso di I/O asincrono, scrivere un programma con istruzioni, successive all'I/O, che dipendano dai dati coinvolti (ad es. letti) durante l'I/O?

SECTION 2: OS161

Domanda 4 (System Call `sys_read` e Sicurezza)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `sys_read`, utilizzata per realizzare la system call `read`. Se ne propone una realizzazione, limitata al caso di standard input.

```
int sys_read(int fd, userptr_t buf, int size) {
    if (fd == STDIN_FILENO) {
        kgets ((char *)buf, size);
        return size;
    } else {
        kprintf("sys_read implemented only on stdio\n");
        return -1;
    }
}
```

Si risponda alle seguenti domande:

-) E' possibile che la funzione sia richiamata direttamente da un processo user? (Motivare la risposta).
 -) La funzione è errata, in quanto la chiamata a `kgets`, così come effettuata, rischia di fare un accesso scorretto in memoria. Si spieghi il perché e si proponga una possibile correzione al problema.
 -) Perché la funzione ritorna `size`, qualora non ci sia errore? E' possibile che, in una realizzazione più completa della funzione, il valore ritornato sia maggiore o minore di `size`?
-

Domanda 5 (Sincronizzazione Multi-core e Semafori)

Contesto: Sia dato un sistema operativo OS161.

Si risponda alle seguenti domande:

-) Si spieghi perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt.
-) Dato codice (ridotto alle parti essenziali) delle funzioni di semaforo P e V, riportate in seguito:

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}

void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
```

```
sem->sem_count++;  
KASSERT(sem->sem_count > 0);  
wchan_wakeone(sem->sem_wchan, &sem->sem_lock);  
spinlock_release(&sem->sem_lock);  
}
```

1. A cosa serve lo spinlock (in entrambe le funzioni)?
 2. Perché la P contiene un ciclo `while` anziché un `if (sem->sem_count == 0)?`
 3. Perché la `wchan_sleep` riceve come parametro lo spinlock?
 4. E' possibile che la chiamata alla `wchan_wakeone` svegli più di un thread in attesa su `wait_channel`?
-) Spiegare brevemente la differenza tra condition variable e `wait_channel`.

ANSWERS

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Dati Iniziali:

- `sizeof(int) = 4` Byte.
- Dimensione pagina = 2 KB = 2048 Byte.
- Frame disponibili per i dati = 10 totali - 1 per le istruzioni = 9 frame.

A) Pagine e frame necessari per i vettori A e B:

Non essendo specificato un indirizzo logico di partenza disallineato, si assumono i vettori allineati al confine di pagina.

- Il vettore A contiene 1024 elementi interi. Dimensione: $1024 \times 4 \text{ B} = 4096 \text{ B} = 4 \text{ KB}$.
- Pagine occupate da A = $4 \text{ KB} / 2 \text{ KB} = \mathbf{2}$ pagine.
- Il vettore B contiene 128×1024 elementi interi. Dimensione: $128 \times 1024 \times 4 \text{ B} = 524.288 \text{ B} = 512 \text{ KB}$.
- Pagine occupate da B = $512 \text{ KB} / 2 \text{ KB} = \mathbf{256}$ pagine.

B) Accessi totali a memoria (Minimo e Massimo):

Le variabili i e j sono mantenute nei registri della CPU.

- **Primo ciclo for:** Esegue esclusivamente `B[j] = 0;`, ovvero 1 scrittura in memoria per iterazione.
Accessi primo ciclo = $128 \times 1024 = \mathbf{131.072}$.
- **Secondo ciclo for (Caso Minimo):** La condizione dell'`if` risulta sempre falsa. Viene eseguita unicamente l'istruzione `j = A[i];` (1 lettura per iterazione).
Accessi secondo ciclo (min) = $1024 \times 1 = \mathbf{1024}$.
Totale Minimo = $131.072 + 1024 = \mathbf{132.096}$ accessi.
- **Secondo ciclo for (Caso Massimo):** La condizione dell'`if` risulta sempre vera. L'istruzione `B[j]++` equivale a `B[j] = B[j] + 1`, richiedendo quindi a livello assembly **1 lettura e 1 scrittura** in memoria. Sommate alla lettura di `A[i]`, si hanno 3 accessi totali per iterazione.
Accessi secondo ciclo (max) = $1024 \times 3 = \mathbf{3072}$.
Totale Massimo = $131.072 + 3072 = \mathbf{134.144}$ accessi.

C) Numero di Page Fault generati (Minimo e Massimo):

Si hanno a disposizione 9 frame fisici per i dati.

- **Analisi Primo Ciclo:** L'inizializzazione sequenziale del vettore B tocca tutte le sue 256 pagine, generando per definizione **256 Page Fault obbligatori**. Al termine del ciclo, nei 9 frame fisici liberi risiederanno le ultime 9 pagine caricate dell'array B.
- **Secondo Ciclo (Caso Minimo):** Si accede unicamente al vettore A. Le 2 pagine di A non sono presenti in RAM. Verranno generati esattamente **2 Page Fault** per ricaricarle, sfrattando 2 pagine di B.
PF Totali Minimi = 256 (Ciclo 1) + 2 (Ciclo 2) = **258 Page Faults**.
- **Secondo Ciclo (Caso Massimo):** Si accede in modo continuo al vettore A e in modo apparentemente casuale al vettore B. Il caricamento di A causa i primi 2 PF. L'algoritmo Second Chance rileverà che le 2 pagine di A vengono lette a *ogni iterazione*, mantenendo il loro *Reference Bit* sempre settato a 1 e impedendone lo sfratto. Di conseguenza, i 7 frame rimanenti saranno contesi dalle pagine di B. Nel caso peggiore, l'indice j punterà sempre a una pagina di B attualmente non caricata nei 7 frame, causando un miss a ogni iterazione (1024 PF massimi per B).
PF Totali Massimi = 256 (Ciclo 1 per B) + 2 (Ciclo 2 per A) + 1024 (Ciclo 2 peggiore per B) = **1282 Page Faults**.

Domanda 2 (SSD e Wear Leveling)

A) Scopo dell'operazione di Wear Leveling:

Questa operazione è opportuna per proteggere l'integrità dei **blocchi fisici** della memoria NAND Flash. Le celle flash supportano un numero limitato di cicli di cancellazione/scrittura (*P/E cycles*) prima di guastarsi. Senza *wear leveling*, un **blocco logico** soggetto a frequentissime sovrascritture (come le tabelle di allocazione del file system) verrebbe scritto sempre nello stesso blocco fisico, causandone la rottura prematura. Il *wear leveling* sfrutta la tabella di mapping per assegnare dinamicamente blocchi fisici sempre nuovi alle scritture del medesimo indirizzo logico, garantendo un'usura omogenea dell'intera superficie del disco.

B) Origine del vantaggio competitivo:

Il vantaggio non deriva da un minor numero di scritture complessive. Al contrario, il meccanismo di wear leveling genera spesso scritture aggiuntive "nascoste" al sistema operativo (poiché il controller sposta in background i dati statici per liberare blocchi poco usurati), innescando il fenomeno della *Write Amplification*. Il vantaggio vitale deriva esclusivamente dall'**equa distribuzione dell'usura**: evitando picchi di stress su celle specifiche, l'SSD massimizza il proprio ciclo di vita utile complessivo.

C) Funzionamento a seguito di blocchi fisici inutilizzabili:

SÌ, il disco continua a funzionare in modo trasparente.

Motivazione: L'architettura degli SSD prevede una quota percentuale di memoria nascosta al sistema operativo, tecnica nota come **Over-Provisioning**. Quando il controller interno (FTL - *Flash Translation Layer*) si accorge che un blocco fisico è danneggiato o ha esaurito i suoi cicli di riscrittura, lo marca in modo permanente come *Bad Block*. Grazie alla tabella di mapping indiretto, il controller reindirizza semplicemente le successive operazioni logiche verso uno dei blocchi fisici di scorta (*spare blocks*) prelevato dall'area di Over-Provisioning, garantendo l'integrità dei dati e la continuità del servizio.

Domanda 3 (I/O Sincrono, Asincrono e Bloccante)

A) Differenze tra tipi di I/O e Sinonimi:

La differenza principale risiede in quando il flusso di controllo ritorna al programma chiamante rispetto al completamento del trasferimento fisico:

- **I/O Sincrono:** La system call ritorna il controllo al processo **solo dopo** che il trasferimento dei dati è stato completato con successo (o è fallito definitivamente).
- **I/O Asincrono:** La system call ritorna **immediatamente**. Il trasferimento dei dati avviene in background (tipicamente delegato al DMA) in parallelo all'esecuzione del processo, che verrà notificato solo ad operazione ultimata.

Bloccante/Non-Bloccante NON sono sinonimi di Sincrono/Asincrono.

Questi due concetti descrivono il comportamento del thread a livello di *scheduling* (in ambito POSIX sono entrambi sottocategorie dell'I/O Sincrono):

- **Bloccante:** Il thread viene sospeso dallo scheduler (stato *Wait/Sleep*) finché l'operazione non è terminata.
- **Non-Bloccante:** La chiamata ritorna immediatamente senza bloccare il thread. Tuttavia, **non avvia un trasferimento in background** (a differenza dell'Asincrono). Se ci sono dati già pronti nel buffer del kernel, li copia e ritorna; se non ci sono, ritorna un errore (es. *EAGAIN*), demandando al processo l'onere di riprovare in futuro.

B) Utilizzo di Polling e Interrupt:

- **I/O Sincrono:** Può essere implementato in **entrambi i modi**. Può usare il Polling (*Programmed I/O*), dove la CPU cicla a vuoto verificando continuamente lo stato del dispositivo (*busy-waiting*), oppure utilizzare gli Interrupt, mettendo il processo a dormire e risvegliandolo a fine operazione.
- **I/O Asincrono:** È **necessario** l'uso degli **interrupt** (spesso con DMA). Poiché la CPU sta eseguendo il codice del processo utente sovrapponendolo all'operazione di I/O, non vi è alcun loop di polling attivo. È il dispositivo hardware che deve obbligatoriamente sollevare un'eccezione (interrupt) per informare il kernel che il buffer in background è pronto.

C) Benefici e dipendenze sui dati:

- **Beneficio:** L'I/O asincrono permette la **sovrapposizione (overlap) totale** tra le fasi di calcolo della CPU (*CPU burst*) e le fasi di accesso alle periferiche (*I/O burst*). Questo massimizza drasticamente il throughput e l'efficienza del processo.
- **Istruzioni successive dipendenti: SÌ, è possibile**, ma tali istruzioni **non possono essere scritte sequenzialmente** subito dopo la system call di invocazione. Devono essere logicamente confinate all'interno di un'apposita routine di gestione del completamento (es. una *Callback Function* o un *Signal Handler*). Il Sistema Operativo garantirà che tale routine venga invocata asincronamente solo dopo la ricezione dell'interrupt, assicurando che i dati letti siano validi e utilizzabili.

Domanda 4 (System Call `sys_read` e Sicurezza)

A) Chiamata diretta da processo user:

NO, non è possibile.

Motivazione: La funzione `sys_read` fa parte del codice del kernel e risiede in un'area di memoria privilegiata (`kseg0`). I processi utente, eseguiti in modalità non privilegiata (*User Mode*), non hanno i permessi hardware per effettuare salti di esecuzione verso tali indirizzi. Per invocare la funzione, il processo utente deve utilizzare l'istruzione speciale `syscall`, la quale genera

un'eccezione (trap) che commuta i privilegi della CPU. Sarà poi il gestore delle eccezioni del kernel a invocare in sicurezza `sys_read`.

B) Accesso scorretto in memoria e correzione:

Spiegazione dell'errore: La funzione prende il puntatore `buf` di tipo `userptr_t` e ne esegue un cast brutale a `char *`, passandolo a `kgets`. Questo è un grave errore di sicurezza. Il kernel non può fidarsi dei puntatori forniti dallo spazio utente. Se un processo malevolo passasse un indirizzo appartenente alla memoria del kernel, l'esecuzione di `kgets` porterebbe alla sovrascrittura e corruzione delle strutture dati interne del sistema operativo.

Correzione: Occorre utilizzare le apposite funzioni di trasferimento sicuro fornite da OS161:

1. Allocare un buffer temporaneo all'interno dello spazio del kernel.
2. Eseguire l'operazione di lettura (`kgets` o affini) caricando i dati in questo buffer temporaneo.
3. Trasferire i dati dal buffer del kernel al buffer utente utilizzando la funzione `copyout()`, la quale esegue autonomamente i controlli di validità sull'indirizzo di destinazione.

C) Valore di ritorno rispetto a `size`:

- **Perché ritorna `size`:** Questa realizzazione di base assume ingenuamente che l'operazione di `kgets` riempi sempre e completamente il buffer per l'intera dimensione richiesta.
- **Può essere minore? SÌ.** In situazioni reali (*Short Read*), l'operazione può restituire un numero di byte inferiore a `size`. Ciò si verifica tipicamente se viene raggiunto l'End Of File (EOF), se l'input da terminale si interrompe al carattere di invio (line buffering), o in caso di lettura da socket di rete parzialmente vuoti.
- **Può essere maggiore? MAI.** Il sistema operativo non deve mai restituire o scrivere un numero di byte superiore al parametro `size` richiesto. Qualora lo facesse, scriverebbe dati oltre i confini della memoria allocata dal processo utente, causando un inevitabile *Buffer Overflow* nello user space.

Domanda 5 (Sincronizzazione Multi-core e Semafori)

A) Inadeguatezza della disabilitazione interrupt nei sistemi Multi-core:

Nei sistemi a singola CPU, disabilitare gli interrupt blocca la *preemption* (lo scheduler non può intervenire), garantendo di fatto la mutua esclusione. In un contesto multi-core, invece, la disabilitazione degli interrupt agisce esclusivamente sul vettore dei flag della **singola CPU locale**. Mentre il core isolato esegue la sua sezione critica, i restanti core mantengono i propri interrupt abilitati e continuano a schedare ed eseguire liberamente altri processi. Tali processi paralleli potrebbero quindi accedere contemporaneamente alle medesime strutture dati condivise, violando la mutua esclusione.

B) Analisi del codice delle funzioni P e V del semaforo:

1. **Ruolo dello spinlock:** Lo spinlock garantisce la mutua esclusione stretta (atomicità) durante l'accesso e la modifica delle variabili interne del semaforo (`sem_count`) e l'ispezione della coda di attesa, prevenendo *race conditions* tra più processi.
2. **Uso del ciclo while (Semantica Mesa):** Il paradigma di sincronizzazione di OS161 utilizza la "Semantica Mesa". Quando un processo viene svegliato tramite `wchan_wakeone`, non viene mandato immediatamente in esecuzione sulla CPU, ma viene semplicemente

reinserito nella coda dei processi pronti (*Ready Queue*). Nel lasso di tempo che intercorre tra il risveglio e l'effettiva esecuzione, un altro processo concorrente potrebbe acquisire la CPU, consumare la risorsa (riportando `sem_count` a 0) e rilasciarla. Il ciclo `while` obbliga il thread appena risvegliato a ricontrollare la condizione, addormentandosi di nuovo se la risorsa non è più disponibile.

3. **Parametro `spinlock` in `wchan_sleep`:** Il passaggio dello `spinlock` è obbligatorio per garantire un'operazione **atomica** di sospensione e rilascio del lock.
 - Se il thread si addormentasse *mantenendo* il lock, causerebbe un **Deadlock** totale (nessuno potrebbe eseguire `V`).
 - Se il thread rilasciasse il lock *prima* di invocare `wchan_sleep`, si esporrebbe al problema del **Lost Wakeup**: un *context switch* prima dell'effettivo addormentamento permetterebbe a un altro processo di eseguire `V` e inviare il segnale di risveglio quando il primo thread non è ancora tecnicamente "dormiente". Il segnale andrebbe perso, e il primo thread dormirebbe per sempre.

Passando lo `spinlock`, la `sleep` lo rilascia internamente nell'esatto momento di invulnerabilità in cui il thread viene definitivamente sospeso.

4. **Wakeone multipli: NO.** La funzione `wchan_wakeone` è progettata per svegliare deterministicamente ed esattamente un solo thread tra quelli in attesa sulla coda.

C) Differenza tra Condition Variable e Wait Channel:

Si tratta di due livelli di astrazione gerarchicamente distinti:

- Il **Wait Channel** (`wchan`) è un costrutto primitivo e di bassissimo livello del kernel. Fornisce unicamente le meccaniche per accodare e mantenere i processi in stato di sonno (bloccati). Non possiede alcuna logica intrinseca di sincronizzazione.
- La **Condition Variable (CV)** è un meccanismo di sincronizzazione di **alto livello**. Permette ai processi di bloccarsi in attesa che si verifichi una specifica espressione logica (condizione applicativa). Per funzionare, la CV incapsula e nasconde al suo interno un `wait_channel` (usato come mera struttura dati per la coda) e deve essere obbligatoriamente associata a un lock esterno per proteggere la valutazione dello stato condiviso.